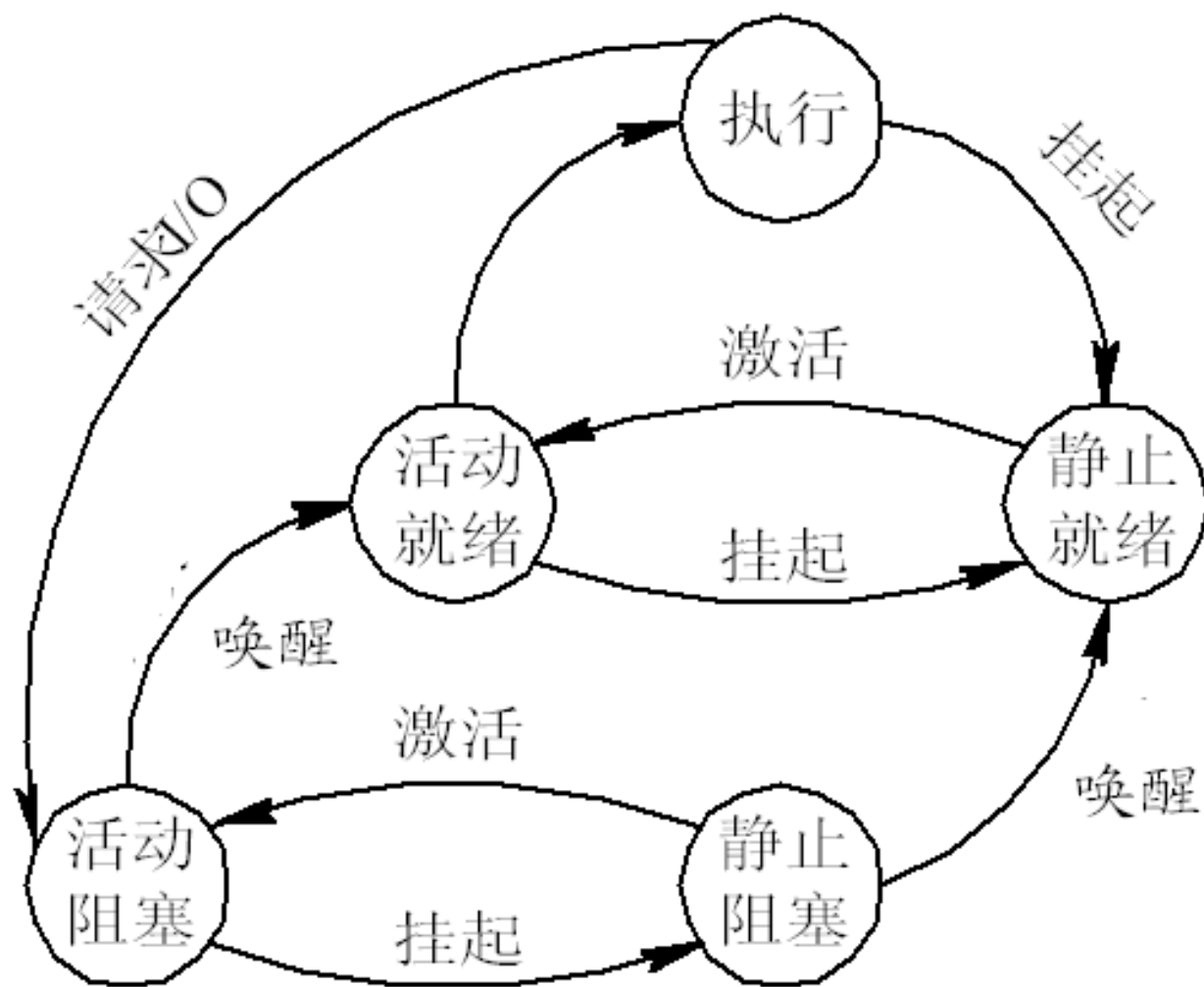


上节回顾

- 前趋图:有向无循环图, 画图方法
- 进程的结构: 程序、数据、控制块PCB
- 进程的特征: 动态性, 并发性, 独立性、异步性
- 进程的定义: 进程是进程实体的**运行过程**, 是系统进行**资源分配和调度**的一个独立单位, P38
- 进程的状态:就绪、执行、阻塞
 - 创建: 有PCB, 缺内存
 - 活动就绪: 获得所有资源, 缺CPU
 - 静止就绪: 缺内存和CPU, 当前的程序数据临时保存在硬盘
 - 执行: 获得CPU执行指令
 - 活动阻塞: 缺CPU、I/O资源
 - 静止阻塞: 缺内存、CPU、I/O资源
 - 终止: 释放所有资源
- 状态转换的原语:P39 图2-6

2.1 进程的基本概念



2.1 进程的基本概念

2.1.5 进程控制块

■ 1. 进程控制块的作用

- ❑ 进程控制块（ **PCB** ， **Process Control Block** ）
- ❑ **PCB**的作用是使一个在多道程序环境下不能独立运行的程序(含数据)，成为一个能独立运行的基本单位，一个能与其它进程并发执行的进程。
- ❑ **OS**是根据**PCB**来对并发执行的进程进行控制和管理。

2.1 进程的基本概念

■ PCB的作用

- 进程存在的唯一标志
- 记录进程的外部特征
- 描述进程变化的过程
- 记录进程和其他进程的联系
- **OS通过PCB控制和管理进程**
 - **OS调度进程进入执行态**
 - 进程间的通信
 - 进程的挂起

2.1 进程的基本概念

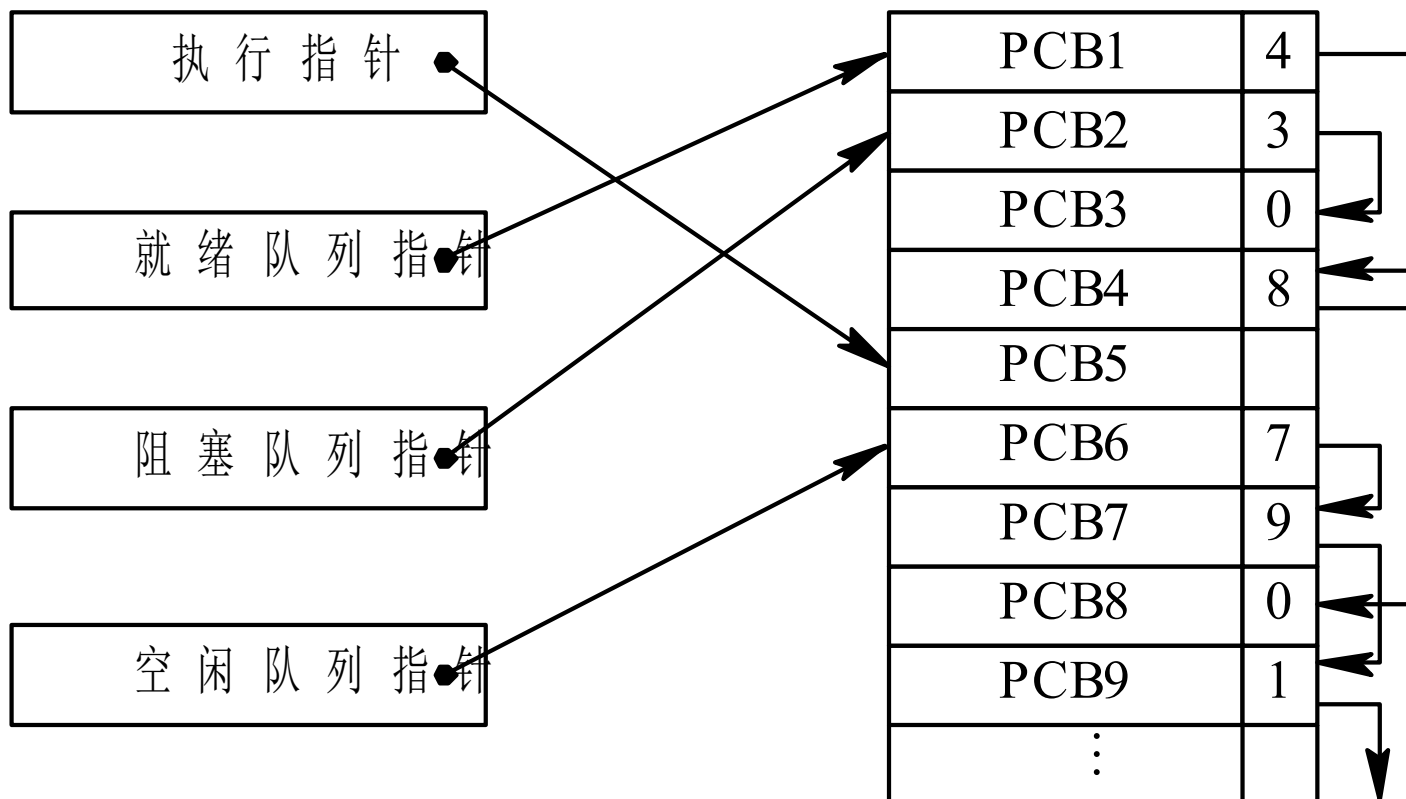
2. 进程控制块中的信息

- 进程标识符：唯一地标识一个进程
 - 内部标识符、外部标识符
- 处理机状态：处理机内各种寄存器的内容
- 进程调度信息：进程调度和进程对换有关的信息
- 进程控制信息：进程资源信息

2.1 进程的基本概念

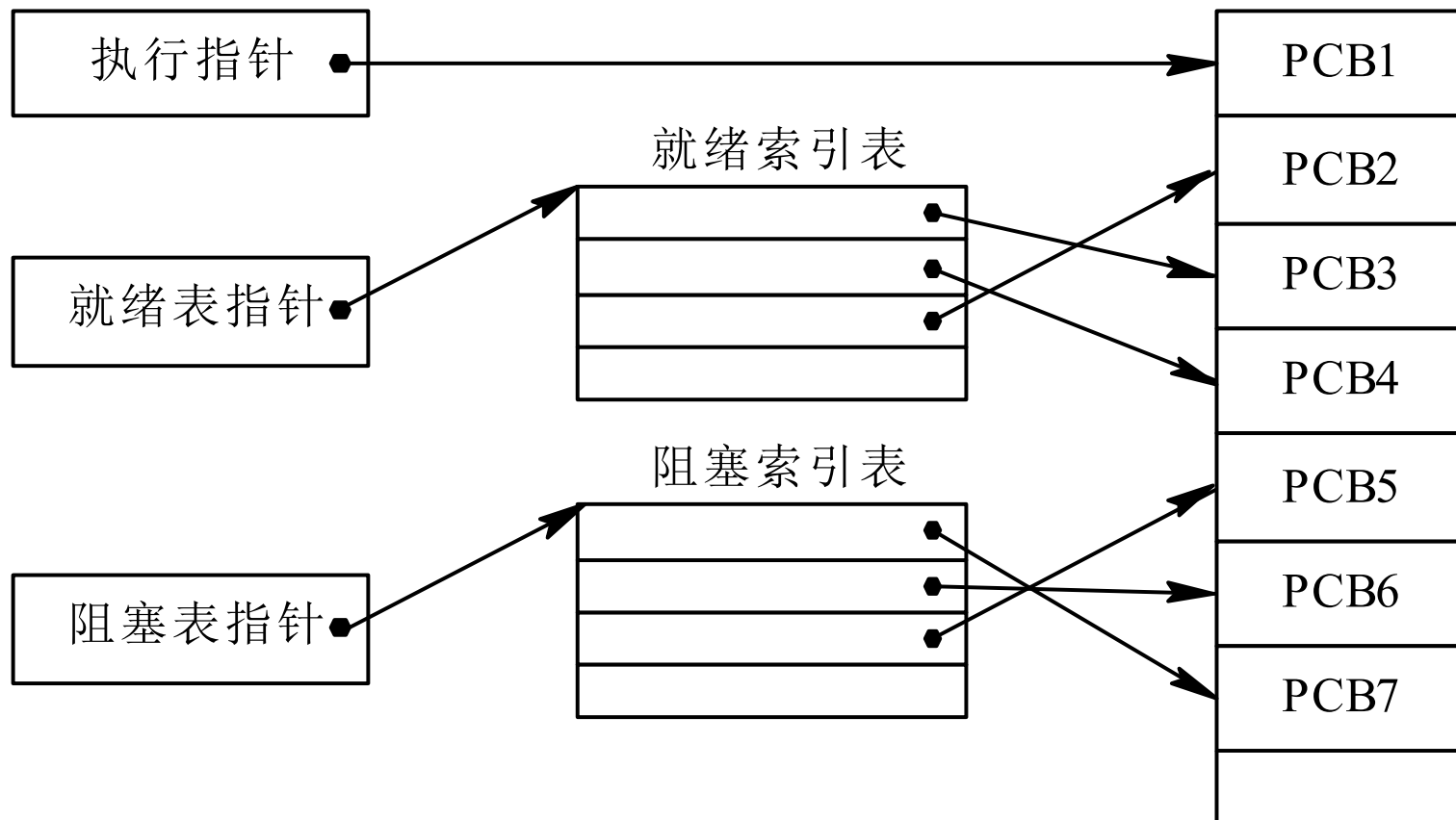
3. 进程控制块的组织方式

■ 链接方式



2.1 进程的基本概念

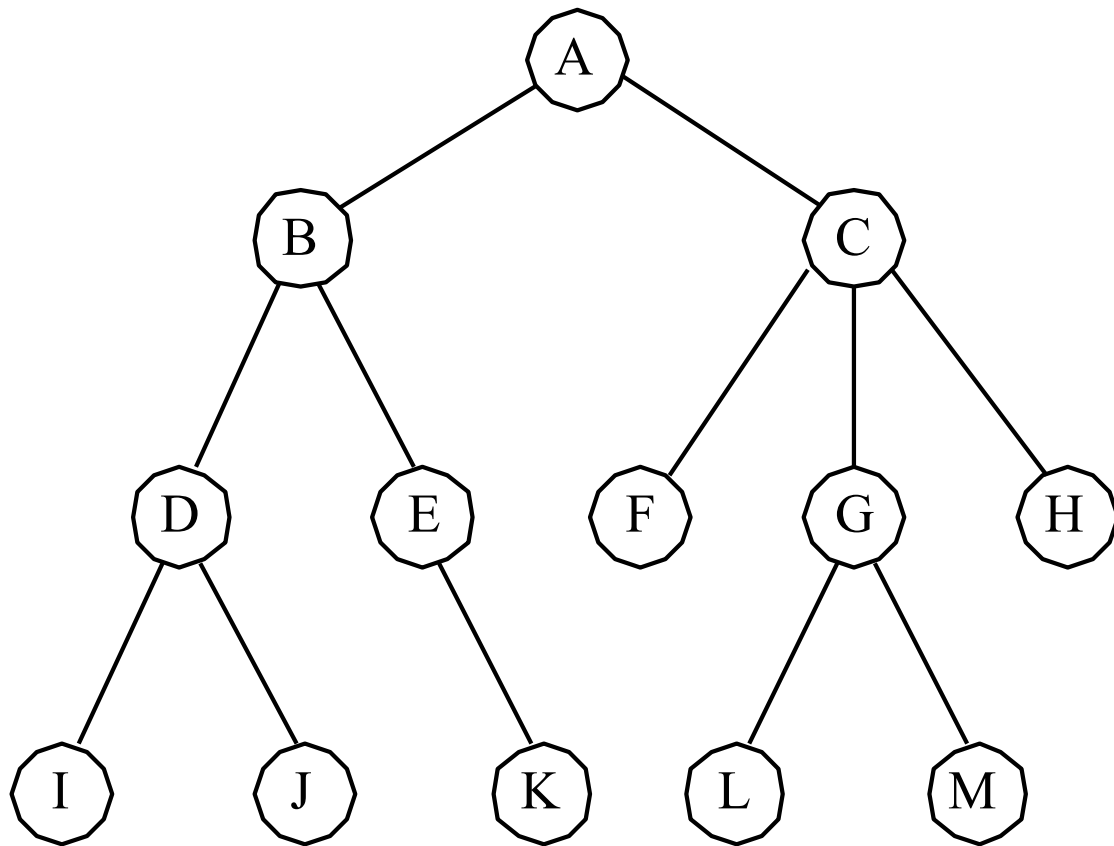
索引方式



2.2 进程控制

2.2.1 进程的创建

- 进程图(**Process Graph**): 描述进程家族关系的有向树



2.2 进程控制

- 父进程：由系统或用户首先创建的进程
- 子进程：父进程创建的进程
- 父子进程关系：
 - 进程控制：子进程由父进程创建或撤销，子进程不能控制父进程
 - 资源继承：子进程可以全部或部分共享父进程的资源
 - 运行方式：可同时进行，可以控制先后
 - **PCB**：在**PCB**中设置家族关系表项，标明自己的父进程和子进程

2.1 进程的基本概念

■ 原语操作

- ❑ 原语本身是由若干个指令组成，用于完成一定功能的一个过程
- ❑ 原语操作是一种原子操作
- ❑ 原语中的操作不可分割，要么全做，要么全不做
- ❑ 原语操作是**OS**内核执行基本操作的主要方法

2.2 进程控制

■ 引起创建进程的事件

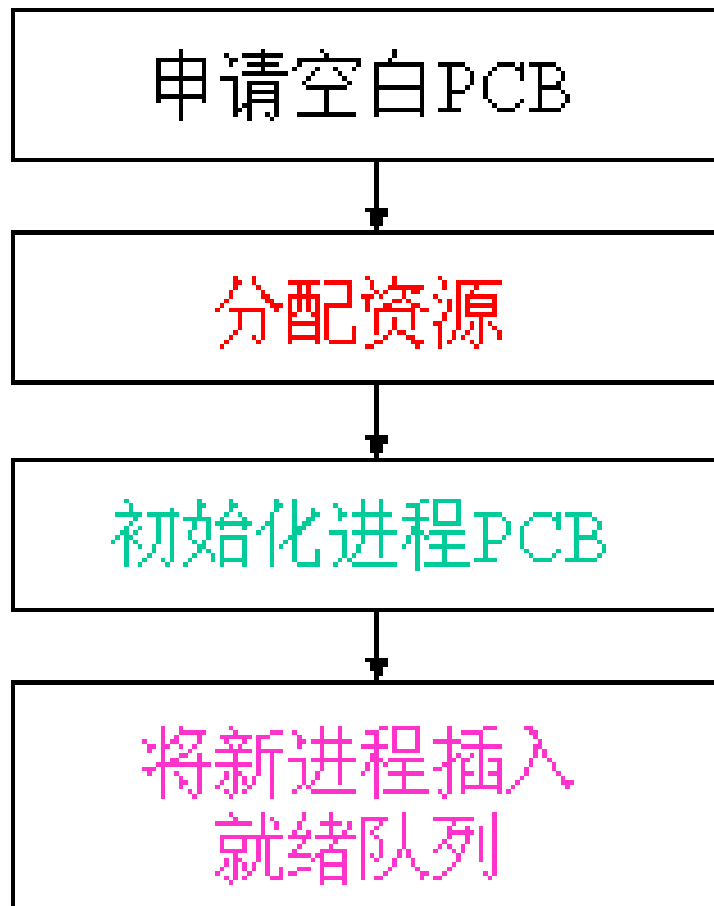
- 用户登录: **winlogon, logon**
- 作业调度。
- 提供服务。
- 应用请求: **Unix的fork()函数, Windows的CreatProcess()函数**

2.2 进程控制

■ 进程的创建

- 申请空白**PCB**。
- 为新进程分配资源：用户请求，自身创建，交互型
- 初始化进程控制块：标识、处理机状态、控制信息。
- 将新进程插入就绪队列，如果进程就绪队列能够接纳新进程，便将新进程插入就绪队列。

2.2 进程控制



2.2 进程控制

2.2.2 进程的终止

■ 1. 引起进程终止的事件

- 正常结束: **halt, Logoff**

- 异常结束

 - 越界错误、保护错、非法指令、特权指令错

 - 运行超时、等待超时、算术运算错、I/O故障

- 外界干预

 - 操作员或操作系统干预

 - 父进程请求

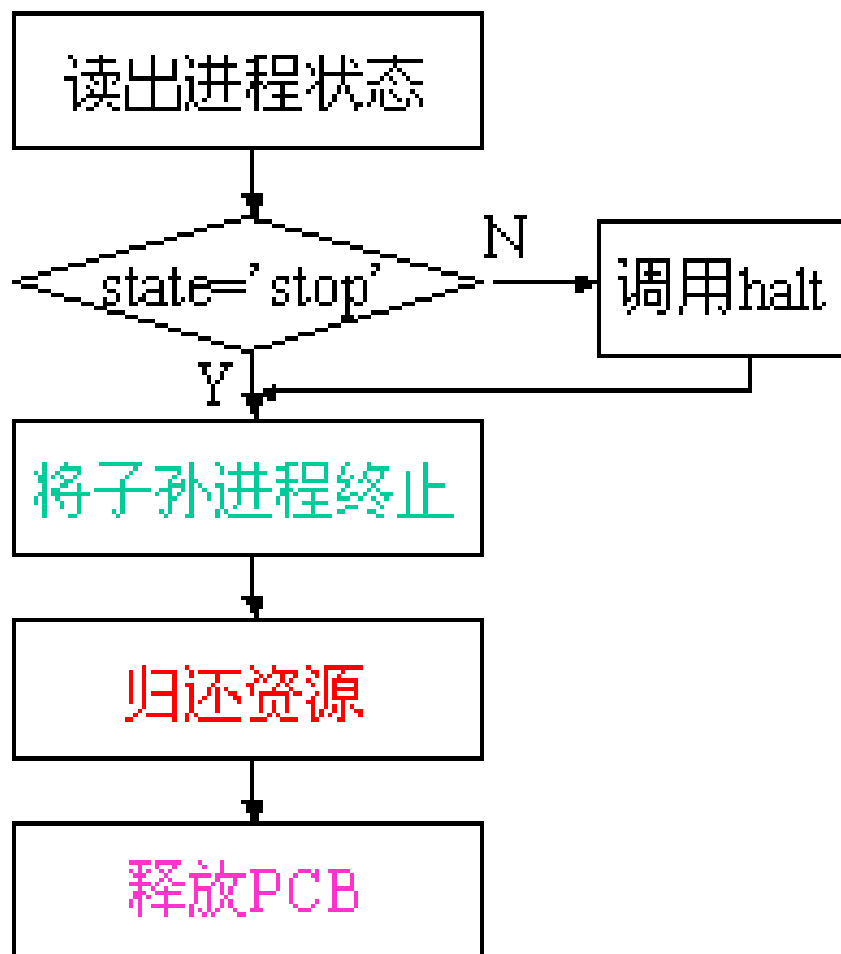
 - 父进程终止

2.2 进程控制

■ 2. 进程的终止过程

- ❑ 根据标识符从**PCB**集合中检索出**PCB**，读取进程状态
- ❑ 若正处于执行状态，立即终止该进程的执行，并置调度标志为真，在进程被终止后重新调度
- ❑ 终止该进程的子孙进程，以防成为不可控的进程
- ❑ 回收资源，或归还给其父进程，或归还给系统
- ❑ **PCB**移出所在队列或链表，等待其他程序来搜集信息

2.2 进程控制



2.2 进程控制

■ LINUX的进程命令

□ ps, ps -u zhangdian, ps -el

□ KILL, KILL -9 324

2.2 进程控制

■ Windows的进程命令

□ TASKLIST, `tasklist -svc`, `tasklist -m`

□ TSKILL

□ NTSD -c q -p PID

- 只有System、SMSS.EXE和CSRSS.EXE不能杀。前两个是纯内核态的，最后那个是Win32子系统，NTSD本身需要它。NTSD从2000开始就是系统自带的用户态调试工具。使用NTSD自动就获得了debug权限，从而能杀掉大部分的进程。

2.2 进程控制

2.2.3 进程的阻塞与唤醒

■ 1. 引起事件

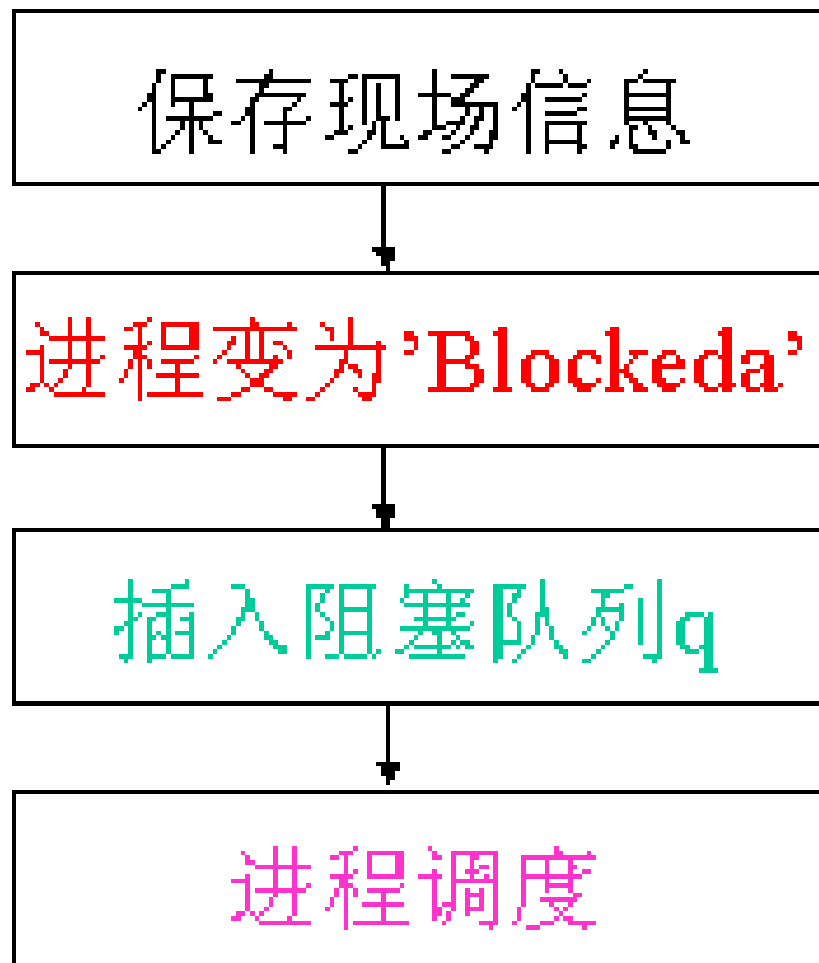
- ☐ 请求系统服务
- ☐ 启动某种操作
- ☐ 新数据尚未到达
- ☐ 无新工作可做

2.2 进程控制

■ 2. 进程阻塞过程

- 正执行过程中，当发现引发阻塞的事件，无法继续执行，进程调用阻塞原语**block**把自己阻塞。
- 阻塞是进程自身的一种主动行为
 - 停止当前执行
 - 修改状态，由执行改为阻塞
 - 将**PCB**插入阻塞队列，若有因不同事件而划分的多个阻塞队列，阻塞进程插入相应队列
 - 重新调度，将处理机分配给另一就绪进程
 - **CPU**环境切换，保存被阻塞进程的**CPU**状态到**PCB**中，按新进程**PCB**的**CPU**状态设置**CPU**新环境

2.2 进程控制

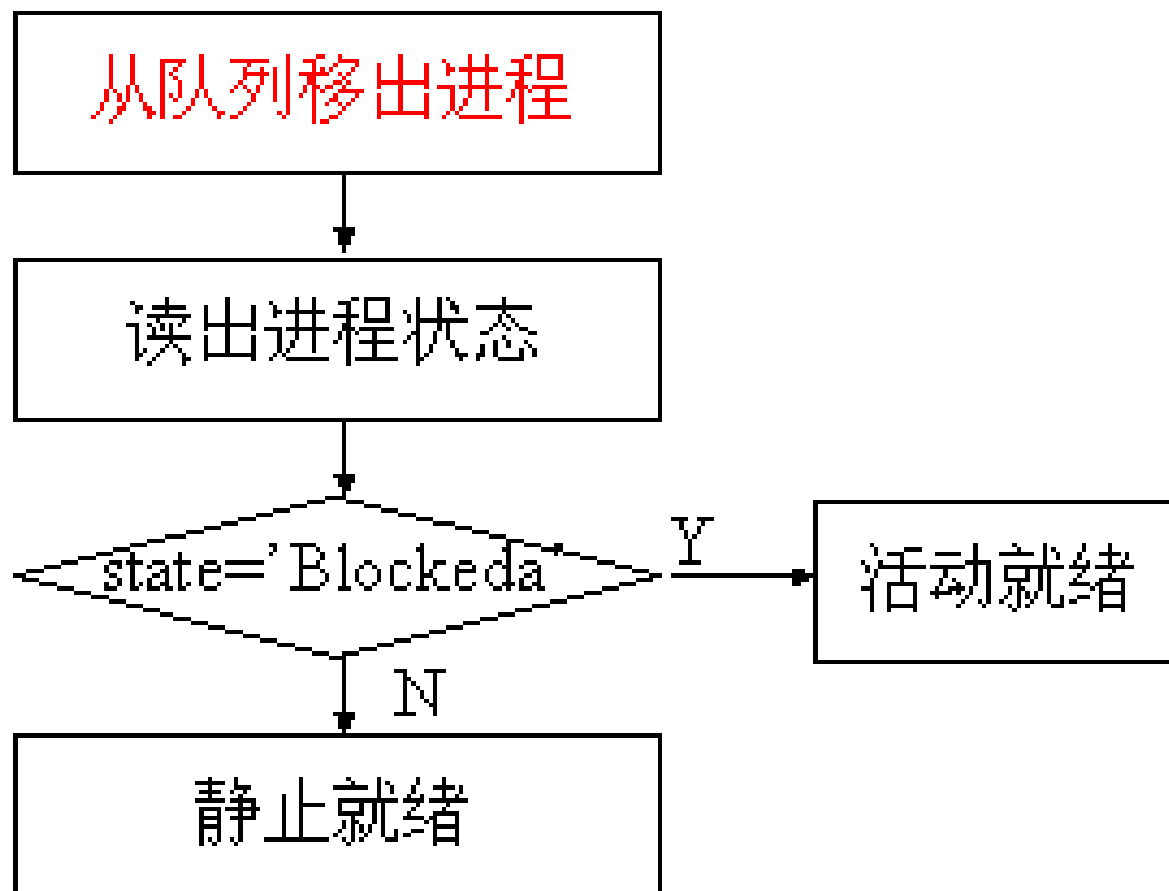


2.2 进程控制

■ 3. 进程唤醒过程

- 被阻塞的事件已满足，如I/O完成或所需数据已到达，则由有关进程(比如，用完并释放了该I/O设备的进程)调用唤醒原语**wakeup**，将等待事件的进程唤醒
- 唤醒原语执行的过程
 - 首先把被阻塞的进程从等待该事件的阻塞队列中移出
 - 将其**PCB**中的现行状态由阻塞改为就绪
 - 再将该**PCB**插入到就绪队列中

2.2 进程控制



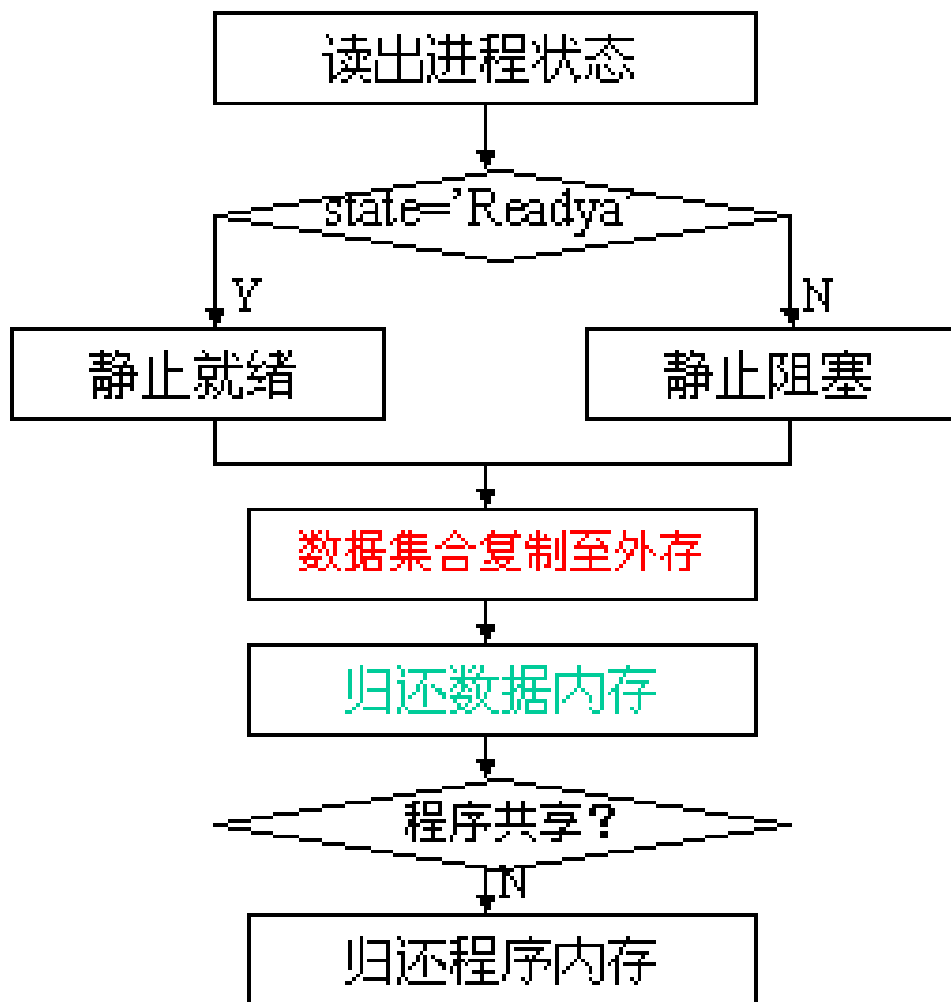
2.2 进程控制

2.2.4 进程的挂起与激活

■ 1. 进程的挂起

- 出现引发挂起的事件（挂起的原因），系统将用原语**suspend**将指定进程挂起
- 挂起原语的执行过程
 - 首先检查被挂起进程的状态，活动就绪改为静止就绪；活动阻塞改为静止阻塞
 - 将**PCB**复制到指定的内存区域
 - 数据复制到外存，释放内存
 - 若被挂起的进程正在执行，则重新调度

2.2 进程控制



2.2 进程控制

■ 2. 进程的激活过程

- 当发生激活进程的事件，如父进程或用户进程请求激活，或当前内存有足够的空间，系统利用激活原语**active**将进程激活。
- 激活原语执行的过程
 - 先将进程从外存调入内存
 - 检查该进程的现行状态，静止就绪改为活动就绪；静止阻塞改为活动阻塞。
 - 若是进入就绪队列，则重新调度，抢占式和非抢占式调度

2.2 进程控制

